Instruction Set Architectures: Accumulator and Stack-Based Machines

---

1. Accumulator Architecture

Concept

An **accumulator architecture** uses a special register called the **accumulator (AC)** to hold intermediate results. All arithmetic and logic operations are performed between the AC and memory operands.

Instruction Set Examples

- `ADD A` $\Rightarrow$ AC := AC + M[A]
- `SUB A` $\Rightarrow$ AC := AC - M[A]
- `MUL A` $\Rightarrow$ AC := AC $\times$ M[A]
- `LOAD A` $\Rightarrow$ AC := M[A]
- `STORE A` $\Rightarrow$ M[A] := AC

Example: $A \times B - (A + B \times C)$

```
LOAD B        ; AC := B
MUL C         ; AC := B * C
ADD A         ; AC := A + B*C
STORE D       ; D := A + B*C
LOAD A        ; AC := A
MUL B         ; AC := A * B
SUB D         ; AC := A*B - (A + B*C)
```

Pros and Cons

Pros

- Minimal hardware requirement
- Simpler instruction format

Cons

- High memory traffic
- Accumulator becomes a bottleneck
- Not suitable for high-speed operations

2. Stack-Based Architecture (Zero-Address Machine)

Concept

Stack-based architecture eliminates the need for explicit operands. Operands are implicitly placed on the top of the stack, and operations pop operands from the stack and push results back.

Instruction Set Examples

- `PUSH A` ⇒ Push A onto stack
- `POP` ⇒ Remove top of stack
- `ADD` ⇒ Pop two operands, add, push result
- `MUL` ⇒ Pop two operands, multiply, push result

Example: $A \times B - (A + C \times B)$

```
PUSH A
PUSH B
MUL         ; A*B
PUSH A
PUSH C
PUSH B
MUL         ; C*B
ADD         ; A + C*B
SUB         ; A*B - (A + C*B)
```

Pros and Cons

Pros

- Very low hardware complexity
- Implicit operand referencing

Cons

- Stack access bottlenecks
- Needs more instructions (e.g. `SWAP`, `POP`)
- Less efficient for complex expressions

3. Summary Table

Feature	Accumulator-Based	Stack-Based (Zero Address)
Operand Source	AC and memory	Top of stack
Instruction Format	One explicit address	No explicit address
Temporary Storage	Accumulator register	Stack
Hardware Complexity	Low	Very low
Instruction Count	Moderate	High
Parallelism	Low	Very low
Efficiency	Medium	Low

Table 1: Comparison of Accumulator vs Stack-Based Architectures

4. Instruction Set Requirements

A good instruction set should be:

- **Complete:** Capable of expressing any computable function.
- **Efficient:** Frequently used operations should be fast and compact.
- **Regular:** Use consistent formats for easier decoding and control.
- **Compatible:** Should support older software and hardware.

5. Classification of Instructions

Type	Purpose	Examples
Data Transfer	Move or copy data between memory and registers	`LOAD`, `STORE`, `MOVE`
Arithmetic	Perform numerical operations	`ADD`, `SUB`, `MUL`
Logical	Bitwise and logical operations	`AND`, `OR`, `NOT`
Program Control	Change flow of execution	`JUMP`, `CALL`, `BRANCH`
Input/Output	Communicate with peripherals	`IN`, `OUT`, `PRINT`

Table 2: Types of Instructions in an ISA